Solution: Simplify Path

Problem Statement

Given an absolute file path in a Unix-style file system, simplify it by converting ".." to the previous directory and removing any "." or multiple slashes. The resulting string should represent the shortest absolute path.

Examples:

```
1.
   Input: "/a//b////c/d//././/.."
   Output: "/a/b/c"

2.
   Input: "/../"
   Output: "/"

3.
   Input: "/home//foo/"
   Output: "/home/foo"
```

Solution

To simplify the path, we'll use a stack to track the directories we're currently in. We'll split the input path into components by the "/" character, then process each component one by one. If a component is "..", we go to the previous directory by popping the top of the stack. If a component is "." or an empty string, we do nothing. Otherwise, we push the component into the stack as a new directory.

Detailed Algorithm steps:

1. Split the input path by the "/" character into an array of components.
2. Initialize an empty stack.
3. For each component in the array:
    - If the component is "..", pop the top of the stack (if it's not already empty).
    - Else if the component is "." or an empty string, do nothing.
    - Else, push the component into the stack as a new directory.
4. Finally, combine the components in the stack into a string, separated by the "/" character. Add a "/" at the start to denote an absolute path.

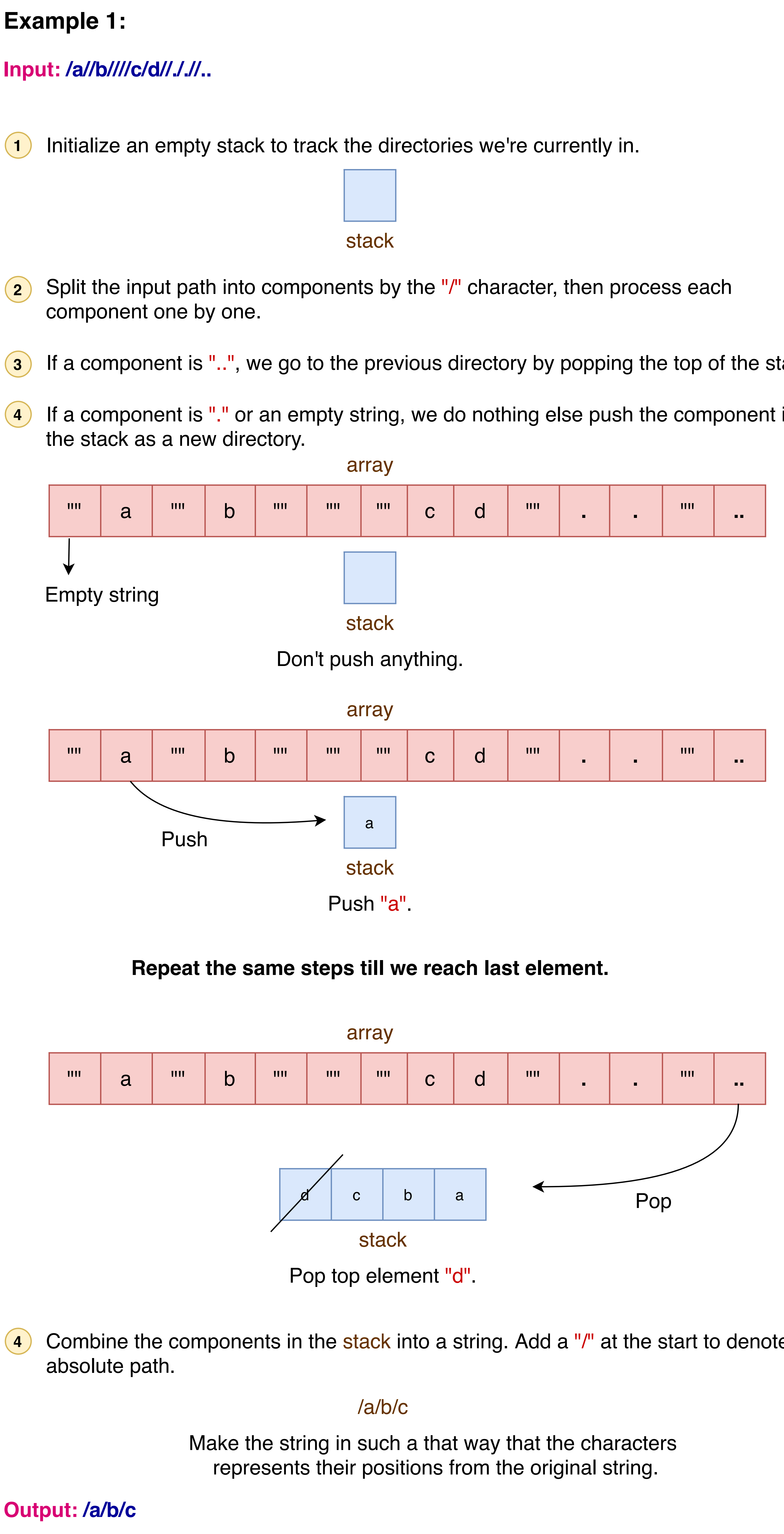
Algorithm walkthrough

let's walk through the code using the input `"/a//b////c/d//././/.."` step by step:

1. **Initialize a Stack**: A `Stack<String>` is created to store components of the simplified path.
2. **Split the Path**: The input path `"/a//b////c/d//././/.."` is split using `"/"` as the delimiter. The resulting parts are: `["", "a", "", "b", "", "", "", "c", "d", "", ".", "", ".", "", "", ".."]`. Empty strings and dots (`"."`) represent the current directory and will be ignored in further processing.
3. **Process Each Part**:
    - **First Part** (`""`): It's empty, so it's ignored.
    - **Second Part** (`"a"`): It's a directory name and is pushed onto the stack.
    - **Third Part** (`""`): Ignored.
    - **Fourth Part** (`"b"`): Another directory name, pushed onto the stack.
    - **Next Several Parts** (`""`, `""`, `""`): All empty, so ignored.
    - **Part** `"c"`: A directory name, pushed onto the stack.
    - **Part** `"d"`: Another directory name, pushed onto the stack.
    - **Part** `"."`: Represents the current directory, ignored.
    - **Part** `"."` **(again)**: Again, represents the current directory, ignored.
    - **Last Part** (`".."`): Represents moving up one directory. It pops `"d"` from the stack.

    After processing, the stack contains: `["a", "b", "c"]`.
4. **Reconstruct Simplified Path**:
    - A `StringBuilder` is used to construct the final simplified path.
    - The stack is processed in LIFO (Last-In-First-Out) order. Each component is popped and appended to the start of the result string, preceded by `"/"`.
    - After processing the stack, the `StringBuilder` contains `"/a/b/c"`.
5. **Return the Result**:
    - The `StringBuilder` is converted to a string and returned.
    - For the given input, the output is `"/a/b/c"`.

Here is the visual representation of the algorithm:



Example 1:

Input: /a//b////c/d//././/..

① Initialize an empty stack to track the directories we're currently in.

② Split the input path into components by the "/" character, then process each component one by one.

③ If a component is "..", we go to the previous directory by popping the top of the stack.

④ If a component is "." or an empty string, we do nothing else push the component into the stack as a new directory.

Don't push anything.

Push "a".

Repeat the same steps till we reach last element.

Pop top element "d".

④ Combine the components in the stack into a string. Add a "/" at the start to denote an absolute path.

/a/b/c

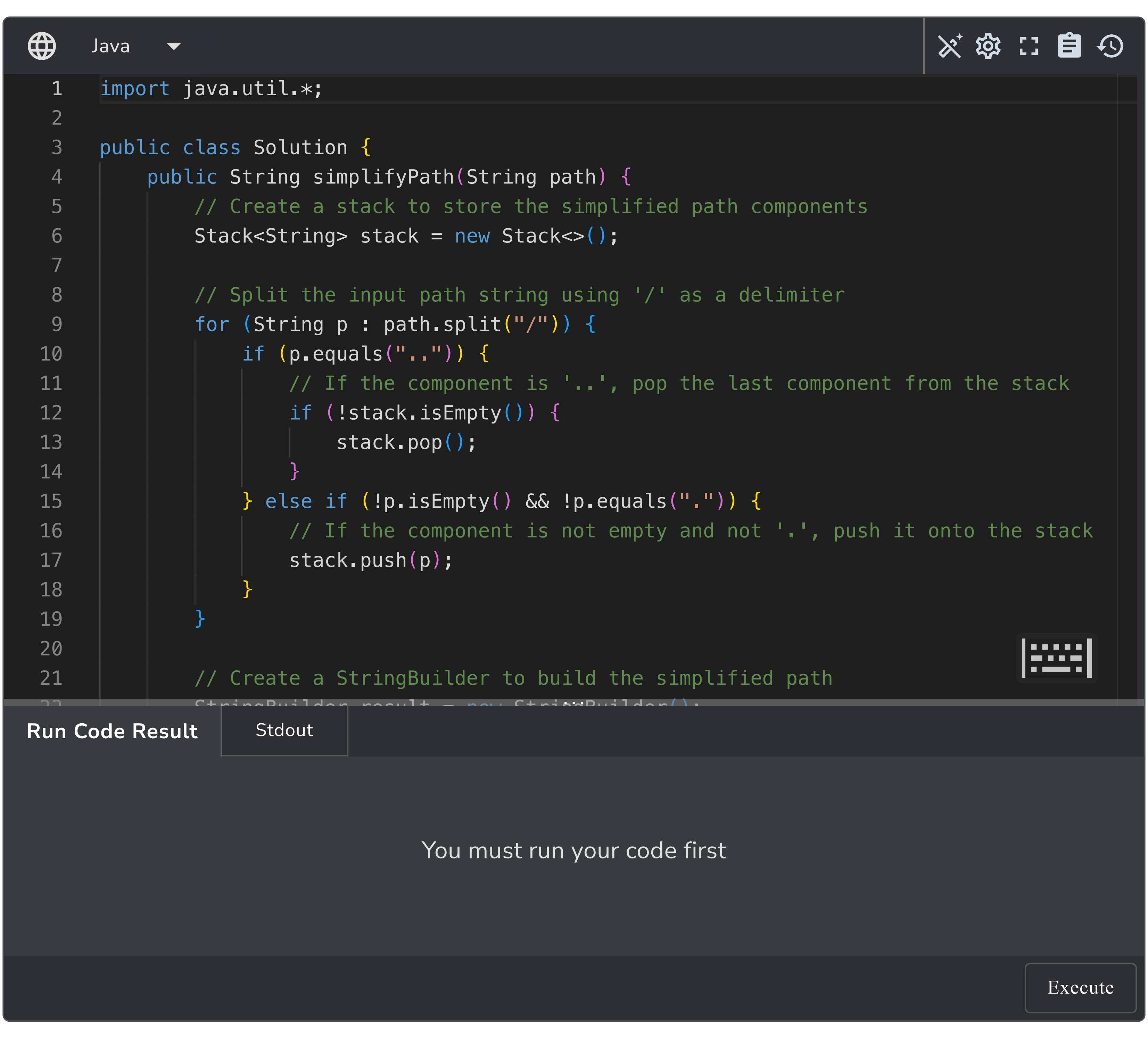
Make the string in such a that way that the characters represents their positions from the original string.

Output: /a/b/c

Simplify Path

Code

Here is the code for this algorithm:



```java
import java.util.*;

public class Solution {
    public String simplifyPath(String path) {
        // Create a stack to store the simplified path components
        Stack<String> stack = new Stack<>();

        // Split the input path string using '/' as a delimiter
        for (String p : path.split("/")) {
            if (p.equals("..")) {
                // If the component is '..', pop the last component from the stack
                if (!stack.isEmpty()) {
                    stack.pop();
                }
            } else if (!p.isEmpty() && !p.equals(".")) {
                // If the component is not empty and not '.', push it onto the stack
                stack.push(p);
            }
        }

        // Create a StringBuilder to build the simplified path
```

You must run your code first

Time and Space Complexity

The time complexity of the algorithm is $O(n)$, where n is the size of the input path, since we process each character once. The space complexity is also $O(n)$, as we store each directory in a stack.

← Previous
Simplify Path (Medium)

Next →
Lowest Common Ancestor Of A Binary Search Tree (Medium)

☐ Mark as Completed